

SECURE MOBILE CROWD SENSING USING BLOCKCHAIN

A PROJECT REPORT

Submitted by

**ANKITA PAL [Reg No: RA1711003010611]
M. SRI ROSHITHA [Reg No: RA1711003010276]**

Under the guidance of

M. ARUL PRAKASH

(Assistant Professor, Department of Computer Science & Engineering)

In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY



**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
S.R.M. NAGAR, KATTANKULATHUR-603 203,
KANCHEEPURAM DISTRICT**

May 2021

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

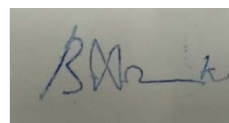
Certified that this project report titled “**SECURE CROWD SENSING USING BLOCKCHAIN** ” is the bonafide work of “**ANKITA PAL [Reg No:RA1711003010611], M. SRI ROSHITHA [Reg No:RA1711003010276]**”, who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.



SIGNATURE

M. ARUL PRAKASH
GUIDE
Assistant Professor
Dept. of Computer Science & Engineering

Signature of the Internal Examiner



SIGNATURE

Dr. B. AMUTHA
HEAD OF THE DEPARTMENT
Dept. of Computer Science & Engineering

Signature of the External Examiner



Own Work Declaration
Department of Computer Science and Engineering

SRM Institute of Science & Technology

Own Work* Declaration Form

This sheet must be filled in (each box ticked to show that the condition has been met). It must be signed and dated along with your student registration number and included with all assignments you submit – work will not be marked unless this is done.

To be completed by the student for all assessments

Degree/ Course : B. Tech. / Computer Science and Engineering

Student Name : Ankita Pal, M Sri Roshitha

Registration Number : RA1711003010611, RA1711003010276

Title of Work : Secure Mobile Crowd Sensing using Blockchain

I / We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism**, as listed in the University Website, Regulations, and the Education Committee guidelines.

I / We confirm that all the work contained in this assessment is my / our own except where indicated, and that I / We have met the following conditions:

- Clearly references / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

I understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:

I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except where indicated by referring, and that I have followed the good academic practices noted above.

If you are working in a group, please write your registration numbers and sign with the date for every student in your group.

RA1711003010611

Ankita Pal

RA1711003010276

M. Sri Roshitha

ABSTRACT

With the recent advancement in computing and sensing capabilities of smart terminals, mobile crowdsensing systems built on smart and ubiquitous mobile terminals have become one of the rising applications allowing a better sensing and scalable systems. However, the prior mobile crowdsensing systems mostly employs centralized architecture, which are subject to low reliability and are vulnerable to malicious attacks. Furthermore, it is difficult to guarantee the privacy with the transparency of MCS systems. These factors have been a major hindrance to the growth of MCS applications. Hence it is imperative to ensure security and privacy of such systems in order to meet the wide spectrum of its' applications. So we propose a decentralized blockchain crowdsensing system to harness the benefits of this sensing paradigm. We plan to integrate blockchain technology into the crowdsensing system. Meanwhile, we also focus on how to protect data privacy in blockchains to develop an accountable MCS system. By leveraging these technologies, we believe that MCS system will have a good performance in privacy protection and security enhancement.

ACKNOWLEDGEMENTS

We express our humble gratitude to C. Muthamizhchelvan, Vice Chancellor (I/C), SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to Dr. Revathi Venkataraman, Professor & Chairperson, School of Computing, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank Dr.B.Amutha, Professor & Head, Department of Computer Science and Engineering, SRM Institute of Science and Technology, for her valuable suggestions and encouragement throughout the period of the project work.

We are extremely grateful to our Academic Advisor Dr.P.Vishalakshi, Assistant Professor, Department of Computer Science and Engineering, SRM Institute of Science and Technology, for their great support at all the stages of project work. We would like to convey our thanks to our Panel Head, Dr.R.Jebakumar, Associate Professor, Department of Computer Science and Engineering, SRM Institute of Science and Technology, for his / her inputs during the project reviews.

We register our immeasurable thanks to our Faculty Advisor, M.Arulprakash, Assistant Professor, Department of Computer Science and Engineering, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to my guide, M.Arulprakash, Assistant Professor, Department of Computer Science and Engineering, SRM Institute of Science and Technology, for providing me an opportunity to pursue my project under his/her mentorship. He / She provided me the freedom and support to explore the research topics of my interest. Her / His passion for solving the real problems and making a difference in the world has always been inspiring.

Ankita Pal

M. Sri Roshitha

TABLE OF CONTENTS

ABSTRACT	iii
ACKNOWLEDGEMENTS	iv
LIST OF FIGURES	vii
ABBREVIATIONS	viii
1 INTRODUCTION	1
1.1 Mobile Crowd Sensing	1
1.2 Summary	3
2 LITERATURE SURVEY	4
2.1 Literature Review	4
2.1.1 General	4
2.1.2 Literature Review	4
2.1.3 Literature Review on Secure Mobile Crowd Sensing	4
3 MOBILE CROWD SENSING	8
3.1 Definition	8
3.2 Background	8
3.3 Summary	9
4 BLOCKCHAIN	10
4.1 Intro	10
4.2 Working	10
4.3 Ethereum Smart Contracts.	11
5 METHODOLOGY	12

5.1	Intro.	12
5.2	Working Procedure	12
5.3	Implementation of Working Procedure	15
6	RESULTS	17
5.1	Intro.	17
5.2	Graphs	18
7	CONCLUSION	22
8	Code and Outputs	23
A.1	Code	23
A.1.1	Defining and Initializing variables	24
A.1.2	Tasks initiated by Requester.	24
A.1.3	Tasks Initiated by Worker	25
A.1.4	Requester Interface	26
A.1.5	Worker Interface	29
A.2	Outputs	31-34
9	References	37

LIST OF FIGURES

A.1	Posting of sensing task by requester	41
A.2	Interface view after the requester clicks the “Create Task”	41
A.3	Submission of data by the worker	41
A.4	Abortion of task by the requester.	42

ABBREVIATIONS

MCS	Mobile Crowd Sensing
GPS	Global Positioning System
DL	Deep Learning
DoS	Denial of Service
BC	Block Chain

CHAPTER 1

INTRODUCTION

Data sensing technology has evolved significantly in recent times as prominence of the data has been realized. As a part of that, fixed-location sensing approaches have become a trend for data collection. Fixed-location sensing approaches include web-based devices which make use of systems like sensors, processors and other hardware devices in order to gather the data from the surroundings, analyse the data collected and transmit the processed data. Though these are helpful in reducing and/or eliminating human intervention, these data sensing approaches doesn't meet the requirements of practical applications as they are constrained due to the less geographical coverage, mobility, high deployment cost and high maintenance cost [1]. Mobile crowdsensing has emerged as a novel data sensing approach to overcome the disadvantages of the above mentioned approach. Mobile terminals are becoming increasingly ubiquitous and crowdsensing has drawn the attention as the next generation sensing technology attributing to its merits that suits the happening informatization era. To be specific, crowdsensing mitigate the deployment cost and provides mobility and scalability.

1.1 Mobile Crowd Sensing

Mobile crowdsensing is a sensing paradigm in which anyone possessing mobile terminals equipped with the robust sensors are capable of sensing the data of common interest [2,3]. This can also be implied as crowdsourcing of sensory data from mobile devices about anything, at any time and from anywhere. In this technique, intelligent mobile terminals equipped with high sensing and computing capabilities are leveraged to gain knowledge and deliver high quality data. Typically, crowdsensing is comprised of three main parties: requesters, workers and a crowdsensing system. The

requesters are sensing task publishers .They issue a sensing task to the system and acquire the sensing data by recruiting the workers. The workers are sensing data contributors. Workers who are interested in the provided task, take up the task from the system and are responsible for collecting and submitting the relevant data to the crowdsensing system. The system is responsible for dealing with the task operations the requester gets the requested data from the system and once the requester is satisfied with the data provided, the worker will be rewarded for his provided service. In specific, MCS extends the functionality of the existing sensing systems without introducing any additional devices.

With the crowdsensing gaining popularity, it has unfolded opportunities for several applications and provided a proper way of assessing the environment (e.g. weather monitoring, noise, air quality), infrastructure (e.g. road condition, traffic analysis) [4] ,social (e.g. healthcare, travel) and many more. Several technology companies are leveraging this technology to provide the services based on the collected data, some of the famous examples being Google and Uber. The robust sensors incorporated in the mobile devices are utilized to collect the pertinent data on a large scale. However, for these applications to be useful in the real-world scenarios there are certain challenges to be dealt with. The immoral exploitation of crowdsensing applications would cause consequential risks. MCS systems have a high extent of malicious security threats, so it is not easy to avoid tampering of data. The prior mobile crowdsensing systems mostly employs centralized architecture where the system is controlled by a central server while such systems are subject to low reliability and are vulnerable to malicious attacks. With centralized architecture, there is also a need to expend on the central platform. MCS have high concern about the privacy too as the sensitive information of the participant may be leaked to the platform for service processing and it is not easy to guarantee the privacy of the users [5,6]. Furthermore, it is imperative to ensure the privacy in order to gratify the participants for their contribution.

1.2 Summary

The contribution of this study is the development of a secure crowdsensing system which integrates the blockchain technology into the mobile crowdsensing. The secure crowdsensing system which we propose, eliminates the need of a central third party in the system and breaks the traditional triangular architecture consequently giving rise to a decentralized MCS system. We implement Ethereum smart contracts deployed on blockchain instead of central server to deal with the operations of the sensing tasks. Ethereum smart contracts, being the irreplaceable codes and the ability of auto-execution on meeting the pre-defined conditions, are leveraged to stimulate the formation of trust fabric in the system and to provide reliable anonymity. This technology can easily mitigate the expense of the central platform and also effectively prevents the system from malicious attacks by making sure that the stored data does not get tampered. We also provide the implementation of our system using the interfaces for both requester task and workers task to access its practicality.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Review

2.1.1 General

Security of crowdsensing applications has become a major concern of the research association and is highly demanded. While it is crucial to understand and adapt to the new sensing technology, it is equally important to embed trust in a potentially untrusted environment because of the high degree of modularity. Also it is hard to set restrictions on the users and data contributors in the crowdsensing applications. Significant works have been carried out to develop the secure applications based on crowdsensing. We briefly explain the related existing work in MCS in this subsection.

2.1.2 Literature Review

The term writing survey is an investigation of a distributed paper, article or other scholarly community. Literature review are not unique substance, and are constantly considered as summarizing work. In the following subsection we portray the remarkable highlights, techniques and contrasts in every one of our reference papers.

2.1.3 Literature Review on Secure Mobile Crowd Sensing

[7] **Kim M, Kim Y. Multi-Blockchain Structure for a Crowdsensing-Based Smart Parking System. Future Internet. 2020; 12(5):90. <https://doi.org/10.3390/fi12050090>**

In [7] the authors propose a multi blockchain based smart parking system with mobile crowdsensing technology in which the public blockchain is used by data contributors and private blockchain is used by the service providers. The bridge node acts as an interface between the two blockchains. The intuition behind the proposed idea is that a huge number of participants can be accommodated while mitigating the

risk of personal exposure. But the proposed scheme have not taken into account the potential security threats of the bridge nodes.

[8] M. Li et al., "CrowdBC: A Blockchain-Based Decentralized Framework for Crowdsourcing," 2019, in **IEEE Transactions on Parallel and Distributed Systems**, vol. 30, no. 6, pp. 1251-1266

CrowdBC [8] is a decentralized framework developed based on blockchain for crowdsensing. It is an Ethereum public test network-based tangible framework which eliminates the need of the conventional centralised system. The proposed framework was successful in mitigating the malicious security attacks like single point of failure by extracting the benefits of blockchains and Ethereum smart contracts. Nonetheless, CrowdBC is not capable of handling the complex situations and hence considered to be impractical in the real time spaces.

[9] S. Zou, J. Xi, H. Wang and G. Xu, "CrowdBLPS: A Blockchain-Based Location-Privacy-Preserving Mobile Crowdsensing System," 2020, in **IEEE Transactions on Industrial Informatics**, vol. 16, no. 6, pp. 4206-4218

CrowdBLPS [9] is a location privacy preserving crowdsensing system which integrated the concept of blockchain into crowdsensing, understanding the need of decentralization to get rid of security issues. Leveraging the concept of smart contract, the proposed scheme has also achieved its' purpose in preserving the privacy of the location and in controlling the data quality by following a two-stage approach. The authors has planned to enhance the reliability of the system to ensure that the system meet the requirements of the real-world.

[10] Jinbo Xiong, Rong Ma, Lei Chen, Youliang Tian, Li Lin, Biao Jin, "Achieving Incentive, Security, and Scalable Privacy Protection in Mobile Crowdsensing Services", 2018, **Wireless Communications and Mobile Computing**

In [10] the authors realized the importance of secure, privacy preserving and incentive crowdsensing services. So they proposed a reliable incentive mechanism for motivating and encouraging the sensing participants to participate in the crowdsensing tasks. They also implemented a privacy preserving scheme by leveraging differential

privacy technique through adding the noise data assuming the participants as not totally credible. Then they used homomorphic encryption for securing the sensing data. In future they plan to concentrate on the tradeoff between accuracy and privacy.

[11] L. Xiao, D. Jiang, D. Xu, W. Su, N. An and D. Wang, "Secure mobile crowdsensing based on deep learning," 2018, in China Communications, vol. 15, no. 10, pp. 1-11

In [11] the authors propose mobile crowdsensing by leveraging the algorithms of deep learning to enhance the security. To thwart security attacks of the crowdsensing systems, the authors investigated the authentication and protection of privacy based on deep learning. The security approaches based on DL are found to be propitious for providing the Quality of Experience based multimedia services. Nonetheless, it was realized that the authentication based on deep neural networks need high training times and the hardware devices used for DL-based computation are not compatible to the MCS devices thus found to be complicated for the practical implementation of mobile crowdsensing systems.

[12] Y. Zhang and B. Kantarci, "Invited Paper: AI-Based Security Design of Mobile Crowdsensing Systems: Review, Challenges and Case Studies," 2019 IEEE International Conference on Service-Oriented System Engineering (SOSE), San Francisco, CA, USA, 2019, pp. 17-1709

In [12] the authors analysed the challenges and the case studies of mobile crowdsensing systems. They have analysed the fact that the illegitimate task requesters are engaged in keeping the servers busy while needlessly expending energy of servers. The proposed scheme focused on the need to differentiate legitimate and illegitimate tasks and leverages the AI-based machine learning techniques in order to provide a solution to the security vulnerabilities of crowd sensing. However, the proposed work is conducted solely on a location-based and energybased DoS attacks and plan to consider more practical scenario in future.

[13] Pius Owoh N, Mahinderjit Singh M. SenseCrypt: A Security Framework for Mobile Crowd Sensing Applications. Sensors (Basel). 2020; 20(11):3280

SenseCrypt [13] is a secure framework that encrypts the sensitive information of the participants of mobile crowdsensing. The proposed framework leveraged the K means algorithm for the clustering of data into sensitive and non-sensitive data. In this the encryption techniques are leveraged in order to secure the sensitive location data of the crowdsensing users. The novelty of the proposed framework is that it has alleviated the cost of computation and communication overhead by leveraging the efficient data compression techniques. However, the paper solely considered the location information as the sensitive data of interest.

[14] Tao Peng, Jierong Liu, Guojun Wang, Qin Liu, Jianer Chen, Jiawei Zhu, "A User-Defined LocationSharing Scheme with Efficiency and Privacy in Mobile Social Networks", 2020, Scientific Programming, vol. 2020, Article ID 7832875, 13 pages

In [14] the authors proposed a custom-defined location sharing strategy for protecting the privacy of the participant in mobile social networks. They considered the vicinity area as a small bounding rectangle for the proposed scheme where the accuracy of the results is affected by the vicinity area. [15] Focused on providing the incentives in a privacy-preserving manner. They aimed for achieving security and privacy protection in MCS.

CHAPTER 3

MOBILE CROWD SENSING

3.1 Definition

Mobile crowd sensing, as the name suggests, simply refers to collecting the sensed data from the people about the surrounding environments by leveraging the mobile sensing terminals. It is a distributed sensing technique that requires a group of individuals possessing mobile sensing devices to sense the surrounding environment and share the information of common interest. Higher the number of participants, higher is the quality of the data obtained and vice versa. Thus, high number of participations is regarded as a key to success of the crowdsensing technique.

3.2 Background

Today, smart devices are equipped with impressive sensor devices such as GPS, accelerometer, microphone and more. Extensive data can be collected with the help of these incorporated sensors. Many research activities have been carried out on collecting data by means of crowdsensing and Crowdsensing has found to be an appealing method to collect data on a large scale and many notable companies have started using this technique in their applications. The process of participatory mobile crowdsensing typically involves three roles: requesters, workers and a processing system. Requesters are the ones who post a sensing task by depositing some amount of rewards for the workers. Workers submit the data in response to the posted data and will be rewarded for submitting the appropriate solution. The processing system act as a bridge between the requester and worker.

Crowdsensing enables the functionality of sensing task by reducing the effort of introducing any additional infrastructure and thereby reducing the cost of maintenance of sensing components. Thus it is a cost-efficient substitute to static sensing

technique. The smart sensors support complex scenarios unlike statically deployed sensors. Mobile crowdsensing helps in seamlessly sensing the data from large groups of individuals at different locations and thus, crowdsensing is scalable. Several application domains can be benefited by using mobile crowdsensing technique. Few of the applications are assessing the environment, transportation, healthcare and many more. Despite the benefits of the technique, it has few disadvantages too. The data contributed during the process is not reliable always as there is a chance of false data being uploaded. Also, the prior and typical crowdsensing system employs centralized framework which is controlled by a central system which are always not reliable and the system can be prone to security attacks. It is also difficult to guarantee the privacy of the participants in crowd sensing.

3.3 Summary

Mobile Crowdsensing serves as a major constituent of the emerging applications of Internet of things. It is a novel sensing technology which takes the advantage of pervasive smart phones and wearable technologies for efficient data collection. This paradigm leverages the powerful sensors embedded in the ubiquitous mobile terminals to collect data on a large scale and is attracting huge interest by reducing the need of hardware requirements and large-scale investments. This trend relies on a huge number of participants to sense and gather the data from the environment through the integrated sensors. Mobile crowdsensing has a structure wherein the contributor manually collects the data using mobile applications and service the user by providing the collected data. MCS does not require an underlying infrastructure. It works by leveraging the mobile devices already used by the users and thus the need for the maintenance of sensor reduces. However, bandwidth and computational power are some of the constraints that limit the potential of crowdsensing. Also, the sensed data can be prone to malicious security and privacy attacks. It is highly imperative to ensure the security and privacy of the data collected through mobile crowdsensing.

CHAPTER 4

BLOCKCHAIN

4.1 Intro

The Blockchain technology uses cryptography and helps it code all the transactions, but you can't exactly see what happened, but you know how it happened. It's one big accounting system or ledger. Instead of being owned by one company, it's owned by everyone. Airbnb, Amazon, EBay, Flipkart, etc.- all these digital platforms that help us facilitate and exchange value; today, we have a technology that allows us to trade one-to-one but on the scale of the entire world, and this is exactly what Blockchain is. We have a network of computer that all have the same ledger, list, information. Thus, there is no requirement of that one institution that stores the data in one master Database. The data gets validated by everyone and it turns to as the next part of the list, constantly updating itself. The technology has become increasingly popular nowadays, and it is also evolving.

4.2 Working

A Blockchain is a distributed ledger that is completely open to anyone. The information stored on a Blockchain is distributed across each member in the network that makes it impossible to change. If a block in the chain was modified, it would be inferred that it had been tampered with. Multiple blocks are chained together where the transactions stored on each block are immutable. Blockchain keeps the data secure through an algorithm known as hashing. With Blockchain being decentralized, it would not need a third party rather, everyone collectively retain control and hence there is no single point of failure. Blockchains are broadly categorized as the public Blockchain and the private Blockchain. When Blockchain technology is integrated into the crowdsensing system, it prevents anyone from censoring the transactions and ensures the data does not get tampered in the network.

4.3 Ethereum Smart Contracts

Ethereum was specifically created and designed to support smart contracts. They can be programmed in a special programming language called Solidity. This language was created mainly for Ethereum. It is a platform that is specifically designed for people to build decentralized applications. The Ethereum clients, or the Ether Browser when combined with Peer-to-Peer networks can be used to create any type of decentralized applications.

Ethereum is popular as it has the following features- security, community support, corporate friendliness, asset issuance, speed, and is uncensored. A smart contract is self-effectuate agreement embedded in digitized code managed by a Blockchain. A set of rules are written into the code under which the parties of the smart contract accept to interact with each other. Once the predefined conditions are met, the smart contract is effectuated automatically. Smart contracts are useful in transferring the digital rewards among the parties. Ethereum enables extending the functionality of Blockchain using smart contracts. Ethereum uses “Proof of Work” which enables the network of Ethereum to accept on the state of information recorded on the Ethereum. A smart contract can be called from other smart contracts within and outside the Blockchain. The use of smart contracts range from simple economic transactions to much more complicated agreements. If smart contracts are implemented in a correct way, they provide the transaction security by preventing certain kinds of economic attacks.

CHAPTER 5

METHODOLOGY

5.1 Intro

We propose a secure Blockchain based crowd sensing framework. The proposed system replaces the conventional triangular architecture by an efficient decentralized framework. The single point of failure attack can be avoided by using the decentralized framework. We leverage the smart contracts deployed on Blockchain technology. The task requesters are initially required to deposit certain amount of money through which the enthusiasm of miners and workers can be raised. By this, certain attacks like Sybil attack, false- reporting attacks can be prevented efficiently. Thus the proposed system provides an advantage of alleviating the malicious attacks and mitigating the charge of the central platform. The privacy can also be guaranteed by allowing the participants to participate without true identity.

5.2 Working Procedure

In a traditional or conventional centralized system, all users or nodes are connected to a central server or network. However, we propose an approach that makes up for the lack of security in the former system. Instead of connecting directly to a server, we will access our application via a web browser, and we will correlate to the client-side app that we will build, a simple app on a web server. This client-side app won't talk to a web backend and database; instead, it'll directly speak to a Blockchain. We will have code written with Ethereum Smart Contract that will contain our pre-conditions and logic for our use case on the Blockchain. That's how a Blockchain works fundamentally and how it's different from a traditional web chain application. A Blockchain is a separate peer to peer network of nodes that talk to one another, a distributed network, and so different computers talking to one another. We can connect to an individual node on the network; our web application is doing it here. All nodes participate in running the network; each node contains all code and data on the Blockchain. The data on the Blockchain has bundles of records. A Blockchain is

secure and unchangeable, making it robust. All code on the Blockchain is on a smart contract, which are the building blocks of Blockchain apps. We will write a Smart Contract that will contain all the tasks as set by the requester. Smart Contracts are written in a programming language called Solidity. Blockchain is trustless, we know no one can change it, and thus the app will behave in the same way every time. Blockchains are like a micro service that executes business logic. We will build a client-side application that will talk directly to the Blockchain and be deployed on it.

Our project uses a decentralized application is essentially a frontend and a smart contract backend combined in one package. It is also very popularly known as the backbone of Web 3.0. The frontend and backend together work as an integrated utility for the entire smart contract. Each user is connected to the decentralized app and is able to obtain and utilize all its features by operating through the front end for easier accessibility. The backend is connected to the Ethereum smart contract in the form of local storage that incorporates into the Blockchain.

The working of a Blockchain maybe split up into the following three categories:

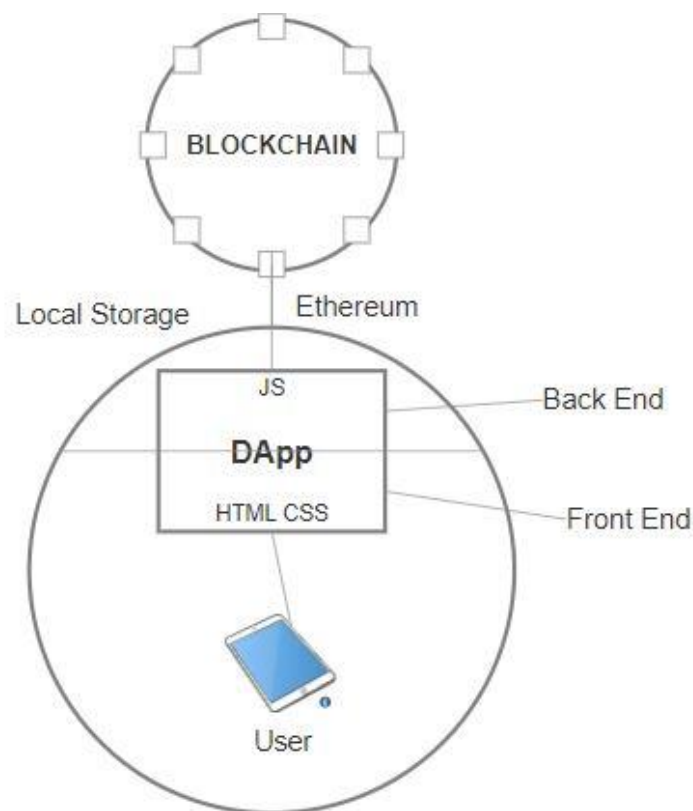
- a. Initiating an innovative smart contract
- b. Writing a smart contract message to a Blockchain
- c. Public key cryptography

Primarily, a public-key encryption infrastructure is used in Blockchain. An initiator coveting to participate in a smart contract hosted on an unauthorized Blockchain can use the position to produce a key and write it on the system. In a conventional Blockchain implementation, when an adequate number of other members or connections arrive at the same outcome, the relevant consent rules determine that the smart contract's information is appended to the Blockchain. Another way is a blockchains administrator may conclude with it.

Decentralized applications or 'DApp' are trustless or peer-to-peer with the differentiating feature that there is no individual server. A DApp uses the Blockchain at the core of its data storage, computation and processing, implemented by smart contracts. A traditional website model creates the User Interface for the decentralized application. After the user accesses the DApp via a web browser, it will connect to the client-side application built on the server. However, the application does not use a

traditional web-based database. Instead, the database is the Blockchain on which the users connected to the Crowd-Sensing Network perform various tasks, including WifiSensing tasks. Inside a DApp, a user's account is a digital Blockchain key stored on the mobile device's memory. Popular tools that help users manage the accounts and interact with the DApp is MetaMask, an extension for web browsers. These are used primarily by the end-users. A suite of software tools used by developers during the construction of a DApp includes integrated development environments (IDEs) such as Remix IDE and the dominant programming language for Ethereum Smart Contracts called Solidity, as used in this use-case.

We introduce smart contracts applications to reward users in our aimed framework, which sensing-task requesters should first launch with particular reserve deposit. There is no centralized stage in the crowd-sensing process in the decentralized system, which we recommend anymore. Alternatively, by implementing Blockchain algorithms, the crowd-sensing process is managed by a decentralized system.



DApp using Blockchain with Ethereum Smart Contracts

5.3 Implementation of Proposed Model

The use case chosen for the project is Road-Sensing. Requesters create the intelligent contract 'Road-Sensing', initiate it, and encrypt sensitive data by private keys. Various parameters and features incorporated into the contract include rewards assigned to each task, the required count of data points, the obtained points, the source and destination of the data point collected, and the task's requester. Several states of the requester's assigned tasks are Uncreated, Created, and Inactive. Additionally, the smart contract denotes multiple conditions for the road sensing task that includes- worst, poor, average, good, and excellent. To better depict the crowd sensing process by the use of smart contracts and Blockchain, we have split into three phases as follows- Abbreviations used in algorithms for the sake of simplicity are R- Requester, W- Worker.

The first algorithm, Algorithm 1, in the implementation deals with creating our Smart Contract and posting the requester's required sensory tasks. The inputs taken in by the algorithm include rewards assigned for the task, the required data for the task, the source as well as the destination for the task. The output will be the status of the completion of the task. The pre-condition set for the algorithm is if the state is Uncreated and if the assigned task's message value is either greater than or equal to the entire product of the reward allocated. Moreover, the required count and the details specified as per the smart contract will be assigned, and the state of the contract becomes 'Created'. Lastly, the status of the contract is returned along with the Task Initiated () function and log.

In Algorithm 2, the Task State and the Obtained Count are parameters of input. The task status, source, destination, and rewards assigned on the road sensing task's registered count are considered output. If the contract's state is 'Created', the worker can only review the task details. When aborting the task, the requester will again check for the contract state's conditions if the amount of obtained count is less than equal to the required count of the data points assigned to the task. If these pre-requisites are met, the contract's state will become 'Inactive'. Finally, the transaction's remaining balance will go to the requester and the Abort Task () function is returned.

The project's final algorithm, Algorithm 3, administers the task's committing and designates awards to the workers after data is uploaded and registers successfully in the Blockchain. The source, destination and road condition is taken in as the input, whereas the average road speed, the obtained data count and the task status are the outputs. In order for the worker to commit the task, the following conditions need to be checked- if the contract's state is 'Created' if the obtained count is less than the required count of the source, as well as the destination of the worker's data, submitted matches the criteria set by the requester. An additional condition that is checked, and finally, if the road's average speed is greater than 10. When all the above-stated conditions are satisfied, the obtained count is incremented by 1. Lastly, when the required count becomes equal to the obtained count, the state is finally changed to 'Inactive', indicating the smart contract process's end. The balance of the transaction is transferred back to the requester and the status is returned. The rewards allocated for the task go to the worker, and the DataCommitted () function is executed.

CHAPTER 6

RESULTS

6.1 Intro

The main aim is to develop a decentralized crowd sensing system for the road sensing application. We outlined the process of mobile crowd sensing by implementing Ethereum smart contracts on Blockchain. Our proposed Road sensing decentralized application was implemented by leveraging Ethereum smart contracts. We used solidity and JavaScript as the programming languages and HTML as the markup language. Solidity is a contract-oriented, object-oriented high-level programming language designed for executing smart contracts on Blockchain platform. Solidity is native to Ethereum Virtual Machine. In addition, we leverage web3.js which allows in designing the decentralized application that interact with the Ethereum Blockchain. Web3.js is an Ethereum API based on JavaScript which provides a way to interact with the Ethereum node.

The usage of smart contracts enhances, intensifies and refurbishes the security aspect of the project because of the following grounds-

1. Smart contracts refer to the immutable computer protocols that digitally aid an agreement's execution. They are a speedier, more economical, more agile and more secure way of administering and operating agreements. Hence, the security of the mobile crowd sensing Blockchain system enhances.
2. The setting of pre-conditions using smart contracts also heightens security. Various modifiers and requirements created and emphasized in the intelligent contract demand the worker's authentication before a task commits and rewards distributed.
3. Cryptography is used to secure transactions in smart contracts to prevent attacks like double spending.

Investigation of the Secure Mobile Crowd-Sensing System using Blockchain In this section, we briefly examines and investigates our crowd sensing based road sensing system from the security aspect.

1. Before participating in the sensing task, participants making the deposits can help in inhibiting several attacks efficiently.
2. By executing smart contracts, only the ETH accounts of the participants (workers and requesters) are known and thus guarantee the anonymity of the participants.
3. A fundamental defect exists in the execution of the project that arises from the loophole of its execution. Intelligent intruders and attackers may use front-run attacking to seize or withdraw partially the rewards meant to provide the workers. For instance, attackers monitoring the network may fetch the workers' committed data and proffer them instantly. The designated worker might not get the expected reward in its entirety if the data's uniqueness has been considered.
4. However, there have been several solutions that may be successful in avoiding the attack—one of those is proclaiming hoaxes.

The implementation of the project as executed successfully on the Remix IDE platform is presented as follows- Requesters create the road sensing task by specifying the input parameters like rewards, number of data required, source and destination and then call `setTask()` function. Requesters have the option to view the amount of data received so far by calling `getDataCnt ()` function. The data count is incremented every time the worker submits the appropriate data. For submitting the collected data,

workers make a call to `commitTask ()` function. Once the preset conditions are met, the worker submits the data and the data count will be incremented by one on successful submission.

6.2 Graphs

The various tables and graphs provided as an exhibit to our project give a detailed insight into our proposed system's transaction and execution costs quantitative analysis.

Transaction costs deal with the expense of transmitting data, and a total of three components account for the transaction cost of the Blockchain's function. They include-

- transaction's base charge
- the smart contract's disposition charge
- the expense of the contract's zero and non-zero bytes of data

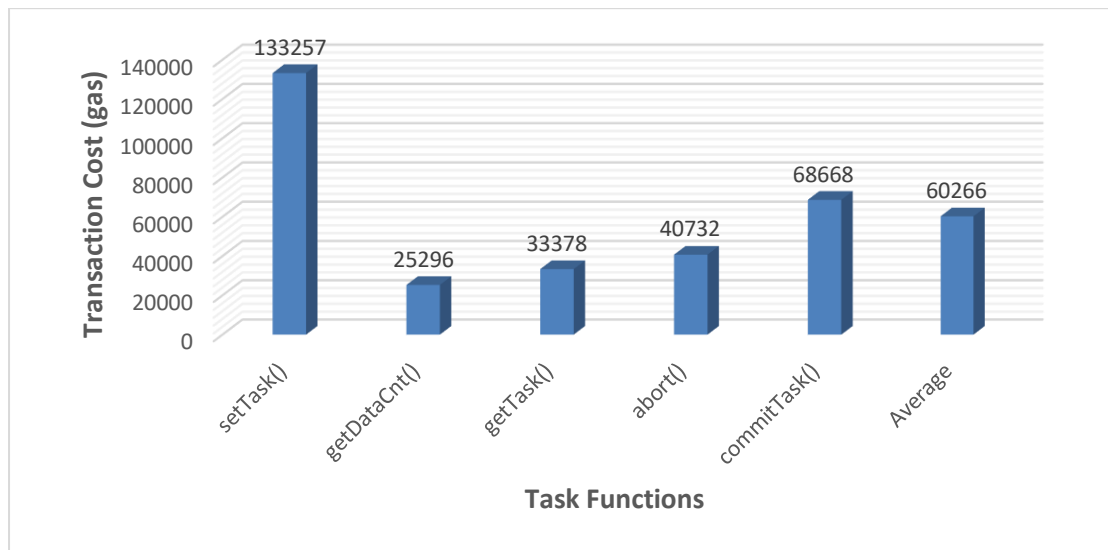
Execution costs include the value of calculation processes, which are performed as a completion of the transaction.

The comprehensive report of the transaction expenses for each task function used in the project has been portrayed. The various task functions used were-- `setTask`, `getDataCnt ()`, `getTask()`, `abort()`, and `commitTask()`. An average for all the costs taken into account is provided for a better estimate of our project's prospects. The transaction costs have been given in terms of the gas being spent on each function.

Task Functions	Transaction Cost (gas)
<code>setTask()</code>	133257
<code>getDataCnt()</code>	25296
<code>getTask()</code>	33378
<code>abort()</code>	40732

commitTask()	68668
Average	60266

Table- Analysis of Transaction Cost for each Task Function

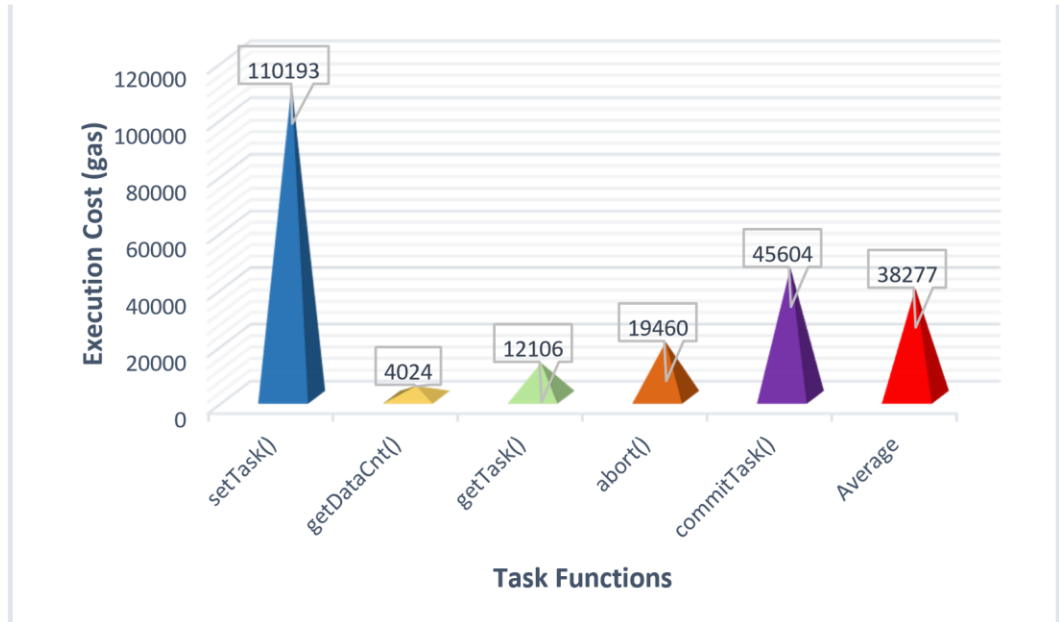


Graph- Analysis of Transaction Costs for each Task Function

The following table and graph give a detailed evaluation of execution costs for each task function that has been performed- setTask(), getDataCnt (), getTask(), abort(), and commitTask().

Task Functions	Execution Cost (gas)
setTask()	110193
getDataCnt()	4024
getTask()	12106
abort()	19460
commitTask()	45604
Average	38277

Table- Evaluation of Execution Costs with respect to Task Functions



Graph- Evaluation of Execution Costs with respect to Task Functions

On analyzing and interpreting this data, we can see that the most expensive function task so far in both transaction and execution costs is the setTask() followed by the commitTask() function. Similarly, the least expensive function task is the getDataCnt ().

Ordering each of the function tasks from being most to least expensive on gas, we get the following as shown in the figure below-

Task Function	Rank of Gas Expensiveness
setTask()	1
commitTask()	2
abort()	3
getTask()	4

getDataCnt()	5
--------------	---

Table- Ordering of Task Functions based on Cost

It can be clearly inferred as referenced from the above two graphs Fig. that the average execution cost is almost only half as much as the average transaction cost. Thus, the total cost incurred for the data calculations is sustained, and hence, the proposed system's efficiency is enhanced. However, the transaction costs may be enhanced further by reducing the base cost of the transactions of each of the task functions.

The objective of this project is to develop a decentralized crowd sensing system that replaces the conventional triangular architecture. We also aimed at leveraging the Ethereum smart contracts by which a successful trade-off between data quality and security preservation can occur.

Hence, both these objectives are satisfied by the execution of our project and proposed model.

CHAPTER 7

CONCLUSION

In conclusion, we proposed an architecture for a decentralized application of the Mobile Participatory Crowd sensing paradigm based road sensing system. We realized that the conventional crowd sensing architecture suffers from single point of failure threat. We leveraged Ethereum Smart Contract deployed on Blockchain technology. The proposed architecture exercises smart contracts that enable crowd sensing providers to proffer their requests and run an action to ascertain the most fitting mobile users engaged in administering the crowd sensing tasks. The framework also employs a reward system for workers and deals with accessing the crowd sensing provider by successfully regulating Blockchain. The proposed architecture implements an incentive-based optimal option for users who handle each crowd sensing task. Hence, we achieve the objectives of the project and enhance the focused parameters of the problem. As a future study, we focus on fully protecting the privacy for the application to meet the real world requirements. We also consider to authenticate the data contributors to avoid data fallacies for getting the reliable data.

APPENDIX A

CODE AND OUTPUTS

A.1 Code

A.1.1 Defining and Initializing variables

```
pragma solidity ^0.4.21;

contract RoadSensing{

    uint public reward;
    uint public requiredCount;
    uint public obtainedCount;
    string public source;
    string public destination;
    address public requester;
    enum State {Uncreated, Created, Inactive}
    State public state;
    enum RoadCondition { Worst, Poor, Average, Good, Excellent}
    RoadCondition public roadcondition;

    mapping(address => bool) public workers;

    modifier condition(bool _condition) {
        require(_condition);
        _;
    }

    modifier onlyRequester() {
        require(msg.sender == requester);
        _;
    }

    modifier inState(State _state) {
        require(state == _state);
        _;
    }

    event TaskInitiated(uint rew, uint reqcnt, string src,string dest);
    event ViewTask(uint rew, uint reqcnt, string src, string dest, uint obtcnt);
    event DataCommitted(string src, string dest, RoadCondition cond, uint avgspd);
    event TaskDone();
    event dataCheck(uint obtcnt);
    event Aborted();
```

A.1.2 Tasks initiated by Requester

```
function setTask(uint _reward, uint _requiredCount, string _source, string _destination)
    public
    inState(State.Uncreated)
    condition(msg.value >= _reward * _requiredCount)
    payable

{
    requester = msg.sender;
    reward = _reward;
    requiredCount = _requiredCount;
    source = _source;
    destination = _destination;
    state = State.Created;
    emit TaskInitiated(_reward, _requiredCount, _source, _destination);
}

function getDataCnt()
    public
    onlyRequester
    inState(State.Created)
{
    emit dataCheck(obtainedCount);
}

function abort()
    public
    onlyRequester
    inState(State.Created)
{
    require(obtainedCount <= requiredCount);
    state = State.Inactive;
    requester.transfer(address(this).balance);
    emit Aborted();
}
```

A.1.3 Tasks Initiated by Worker

```
function getTask()
  public
  inState(State.Created)
{
  emit ViewTask(reward, requiredCount, source, destination, obtainedCount);
}

function hashCompareWithLengthCheck(string a, string b) internal returns (bool)
{
  if(bytes(a).length != bytes(b).length) {
    return false;
  } else {
    return (keccak256(abi.encodePacked((a))) == keccak256(abi.encodePacked((b))));
  }
}

function commitTask(string _source, string _destination, RoadCondition _roadcondition, uint _averageSpeed)
  public
  inState(State.Created)
{
  require(!workers[msg.sender]);
  require(obtainedCount < requiredCount);
  require(hashCompareWithLengthCheck(source,_source));
  require(hashCompareWithLengthCheck(destination,_destination));
  require(_averageSpeed > 10);

  obtainedCount += 1;
  if(obtainedCount == requiredCount){
    state = State.Inactive;
    requester.transfer(address(this).balance);
    emit TaskDone();
  }
  workers[msg.sender] = true;
  msg.sender.transfer(reward);
  emit DataCommitted(_source, _destination, _roadcondition, _averageSpeed);
}
}
```

A.1.4 Requester Interface

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <meta http-equiv="X-UA-Compatible" content="ie=edge">
7   <title>Road Sensing</title>
8
9   <link rel="stylesheet" type="text/css" href="main.css">
10
11   <script src="./node_modules/web3/dist/web3.min.js"></script>
12
13 </head>
14 <body>
15   <div class="container">
16
17     <h1>Road Sensing (Requester)</h1>
18
19     <h2 id="init"></h2>
20     <span id="Trans"></span>
21     <hr>
22
23     
24     <label for="reward" class="col-lg-2 control-label">Value of Each Reward</label>
25     <input id="reward" type="text">
26
27     <label for="requiredCount" class="col-lg-2 control-label"># of Data Required</label>
28     <input id="requiredCount" type="text">
29
30     <label for="source" class="col-lg-2 control-label">Source</label>
31     <input id="source" type="text">
32
33     <label for="destination" class="col-lg-2 control-label">Destination</label>
34     <input id="destination" type="text">
35
36     <button id="button1">Create Task</button>
37     <button id="button2">Abort Task</button>
38     <button id="button3">Check DataCount</button>
39   </div>
40
41   <script src="https://code.jquery.com/jquery-3.2.1.slim.min.js"></script>
42
43   <script>
44     if (typeof web3 !== 'undefined') {
45       web3 = new Web3(web3.currentProvider);
46     } else {
47       // set the provider you want from Web3.providers
48       web3 = new Web3(new Web3.providers.HttpProvider("http://localhost:8545"));
49     }
50
51     web3.eth.defaultAccount = web3.eth.accounts[0];
52
53     var content = '';
54
55     var RoadSensing = web3.eth.contract('ENTER THE ABI ADDRESS HERE');
56     var sensingRoadContract = RoadSensing.at('ENTER THE CONTRACT ADDRESS');
57     console.log(sensingRoadContract);
58   </script>
```

```

58
59     var initEvent = sensingRoadContract.TaskInitiated({}, 'latest');
60
61     initEvent.watch(function(error, result){
62         if (!error){
63             $("#loader").hide();
64             $("#Trans").html('Block hash: ' + result.blockHash);
65             $("#init").html(result.args.reqcnt.c+' data about the road from \''
66                 +result.args.src+'\'' to \''+result.args.dest+'\'' are required,
67                 and each of them is rewarded by '+result.args.rew.c+ ' wei.');
```

```

68         } else {
69             $("#loader").hide();
70             console.log(error);
71         }
72     });
73
74
75     var abortEvent = sensingRoadContract.Aborted({}, 'latest');
76
77     abortEvent.watch(function(err, res){
78         if (!err){
79             $("#loader").hide();
80             $("#Trans").html('Block hash: ' + res.blockHash);
81             $("#init").html('Money has been withdrawn!');
```

```

82         } else {
83             $("#loader").hide();
84             console.log(err);
85         }
86     });
87
88     var checkCountEvent = sensingRoadContract.dataCheck({}, 'latest');
89
90     checkCountEvent.watch(function(error, result){
91         if (!error){
92             $("#loader").hide();
93             $("#Trans").html('Block hash: ' + result.blockHash);
94             $("#init").html(result.args.obtcnt.c+' data have been collected.');
```

```

95         } else {
96             $("#loader").hide();
97             console.log(error);
98         }
99     });
100
101     var dataCollected = sensingRoadContract.DataCommitted({}, 'latest');
102
103     dataCollected.watch(function(error, result){
104         if (!error){
105             $("#loader").hide();
106             content = '<br>Source: '+result.args.src+', Destination: '
107                 +result.args.dest+', Average Speed: '+result.args.avgspd.c;
108             $("#init").html(content);
109         } else {
110             $("#loader").hide();
111             console.log(error);
112         }
113     });
114

```

```

$("#button1").click(function() {
    $("#loader").show();
    sensingRoadContract.setTask($("#reward").val(),
    $("#requiredCount").val(), $("#source").val(),$("#destination").val(),
    {value: $("#reward").val()*$("#requiredCount").val()+10000000000000000},(err, res) => {
        if (err) {
            $("#loader").hide();
        }
    });
});

$("#button2").click(function() {
    $("#loader").show();
    sensingRoadContract.abort((err, res) => {
        if (err) {
            $("#loader").hide();
        }
    });
});

$("#button3").click(function() {
    $("#loader").show();
    sensingRoadContract.getDataCnt((err, res) => {
        if (err) {
            $("#loader").hide();
        }
    });
});
</script>
</body>
</html>

```

A.1.5 Worker Interface

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <meta http-equiv="X-UA-Compatible" content="ie=edge">
7   <title>Road Sensing</title>
8
9   <link rel="stylesheet" type="text/css" href="main.css">
10
11   <script src="./node_modules/web3/dist/web3.min.js"></script>
12
13 </head>
14 <body>
15   <div class="container">
16
17     <h1>Road Sensing (Worker)</h1>
18
19     <h2 id="init"></h2>
20     <span id="Trans"></span>
21     <hr>
22
23     
24
25
26     <label for="source" class="col-lg-2 control-label">Source</label>
27     <input id="source" type="text">
28
29     <label for="destination" class="col-lg-2 control-label">Destination</label>
30     <input id="destination" type="text">
31
32     <label for="roadcondition" class="col-lg-2 control-label">Road Condition</label>
33     <input id="roadcondition" type="text">
34
35     <label for="averagespeed" class="col-lg-2 control-label">Average Speed</label>
36     <input id="averagespeed" type="text">
37
38     <button id="button1">View Task</button>
39     <button id="button2">Commit</button>
40
41   </div>
42
43   <script src="https://code.jquery.com/jquery-3.2.1.slim.min.js"></script>
44
45   <script>
46     if (typeof web3 !== 'undefined') {
47       web3 = new Web3(web3.currentProvider);
48     } else {
49       // set the provider you want from Web3.providers
50       web3 = new Web3(new Web3.providers.HttpProvider("http://localhost:8545"));
51     }
52
53     web3.eth.defaultAccount = web3.eth.accounts[0];
54
55     var content = '';
56
```

```

57 var RoadSensing = web3.eth.contract(ENTER ABI ADDRESS HERE);
58 var sensingRoadContract = RoadSensing.at('ENTER CONTRACT ADDRESS HERE');
59 console.log(sensingRoadContract);
60
61 var viewTask = sensingRoadContract.ViewTask({}, 'latest');
62
63 viewTask.watch(function(error, result){
64     if (!error){
65         $("#loader").hide();
66         $("#Trans").html('Block hash: ' + result.blockHash);
67         $("#init").html(result.args.reqcnt.c+' data about road condition from \''
68 +result.args.src+'\" to \''+result.args.dest+'\" are required,
69 and each of them is rewarded by '+result.args.rew.c+ ' wei.<br> Till now, '
70 +result.args.obtcnt.c+'/'+result.args.reqcnt.c+' of data were collected.');
```

```

71     } else {
72         $("#loader").hide();
73         console.log(error);
74     }
75 });
76
77 var doneEvent = sensingRoadContract.TaskDone({}, 'latest');
78
79 doneEvent.watch(function(error, result){
80     if (!error){
81         $("#loader").hide();
82         $("#Trans").html('Block hash: ' + result.blockHash);
83         $("#init").html('Task has been closed.');
```

```

84     } else {
85         $("#loader").hide();
86         console.log(error);
87     }
88 });
89
90 var commitEvent = sensingRoadContract.DataCommitted({}, 'latest');
91
92 commitEvent.watch(function(error, result){
93     if (!error){
94         $("#loader").hide();
95         $("#Trans").html('Block hash: ' + result.blockHash);
96         $("#init").html('Data has been successfully submitted! <br>'
97 + ' Source: '+result.args.src+', Destination: '+result.args.dest+',
98 Road Condition: '+result.args.cond.c+', Average Speed: '+result.args.avgspd.c+' kmph.');
```

```

99     } else {
100         $("#loader").hide();
101         console.log(error);
102     }
103 });
104
105 $("#button1").click(function() {
106     $("#loader").show();
107     sensingRoadContract.getTask((err, res) => {
108         if (err) {
109             $("#loader").hide();
110         }
111     });
112 });

```

A.2 Outputs

A.2.1 Requester posting the task

The screenshot displays a web application interface for managing transactions. On the left, a sidebar titled 'DEPLOY & RUN TRANSACTIONS' contains buttons for 'abort', 'commitTask', 'getDataCnt', 'getTask', 'setTask', 'destination', 'obtainedCount', 'requester', 'requiredCount', 'reward', 'roadcondition', 'source', and 'state'. The 'setTask' button is highlighted in red. The main area shows a transaction details view for a 'setTask' operation. The transaction is sent to the address '0xd9145ccf520386f254917e481e844c9943f39138'. The gas used is 3000000, the transaction cost is 133257 gas, and the execution cost is 110193 gas. The input is a JSON object:

```
{ "uint256_reward": { "type": "BigNumber", "hex": "0x05" }, "uint256_requiredCount": { "type": "BigNumber", "hex": "0x04" }, "string_source": "CHN", "string_destination": "MUM" }
```

. The decoded output is an empty array. The logs show a 'TaskInitiated' event with arguments:

```
{ "0": "5", "1": "4", "2": "CHN", "3": "MUM", "rew": "5", "reqcnt": "4", "src": "CHN", "dst": "MUM" }
```

. The value is 30 wei.

Figure A.1: posting of sensing task by requester

A.2.2 Interface view of requester posting a task

Road Sensing (Requester)

2 data about the road from "mum" to "chn" are required, and each of them is rewarded by 1 wei.

Block hash: 0xf79d070a8dfc3511c3d4808459c85b708f595e2d36357c1f073548505fc10fda

Value of Each Reward

of Data Required

Source

Destination

Create Task

Abort Task

Check DataCount

Figure A.2: Interface view after the requester clicks the “Create Task”

A.2.3 Worker committing the required data

The screenshot displays a web application interface for managing transactions. On the left, a sidebar contains icons for various actions: a blue circle with a white 'H', a document icon, a gear icon, a magnifying glass icon, and a list icon. The main area is titled 'DEPLOY & RUN TRANSACTIONS' and features a list of transactions. The transactions are listed in a table with columns for the transaction name, its parameters, and its status. The transactions are: 'abort', 'commitTask' (with parameters 'CHN', 'MUM', 3, 60), 'getDataCnt', 'getTask', 'setTask' (with parameters 5, 3, 'CHN', 'MUM'), 'destination', 'obtainedCount', 'requester', 'requiredCount', 'reward', 'roadcondition', 'source', and 'state'. The 'setTask' transaction is highlighted in red. On the right, a detailed view of a transaction is shown, including its status, transaction hash, from, to, gas, transaction cost, execution cost, hash, input, decoded input, and decoded output. The transaction is titled 'transact to RoadSensing.commitTask pending ...'. The status is 'True Transaction mined and execution succeed'. The transaction hash is '0x88e463758d7a89c656083d1ab0c1d89d760b264bab967dd48512861fa2e135'. The from address is '0x58380a6a701c568545dcf803fc8875f56bddd4'. The to address is 'RoadSensing.commitTask(string,string,uint8,uint256)'. The gas is '3000000 gas'. The transaction cost is '68568 gas'. The execution cost is '45604 gas'. The hash is '0x88e463758d7a89c656083d1ab0c1d89d760b264bab967dd48512861fa2e135'. The input is '0xb15...00000'. The decoded input is '{ "string_source": "CHN", "string_destination": "MUM", "uint8_roadcondition": 3, "uint256_averagespeed": { "type": "bignumber", "hex": "0x3c" } }'. The decoded output is '()'. A 'Debug' button is visible next to the transaction details.

Transaction	Parameters	Status
abort		
commitTask	"CHN","MUM",3,60	
getDataCnt		
getTask		
setTask	5,3,"CHN","MUM"	
destination		
obtainedCount		
requester		
requiredCount		
reward		
roadcondition		
source		
state		

transact to RoadSensing.commitTask pending ...

[vm] from: 0x583...eddC4
to: RoadSensing.commitTask(string,string,uint8,uint256) 0xd91...39138 value: 0 wei
data: 0xb15...00000 logs: 1 hash: 0x88e...2e135

status: True Transaction mined and execution succeed

transaction hash: 0x88e463758d7a89c656083d1ab0c1d89d760b264bab967dd48512861fa2e135

from: 0x58380a6a701c568545dcf803fc8875f56bddd4

to: RoadSensing.commitTask(string,string,uint8,uint256)
0xd9145CC52D386f254917e481c044e9943f39138

gas: 3000000 gas

transaction cost: 68568 gas

execution cost: 45604 gas

hash: 0x88e463758d7a89c656083d1ab0c1d89d760b264bab967dd48512861fa2e135

input: 0xb15...00000

decoded input: { "string_source": "CHN", "string_destination": "MUM", "uint8_roadcondition": 3, "uint256_averagespeed": { "type": "bignumber", "hex": "0x3c" } }

decoded output: ()

Figure A.3: Submission of data by the worker

A.2.4 Requester aborting the task

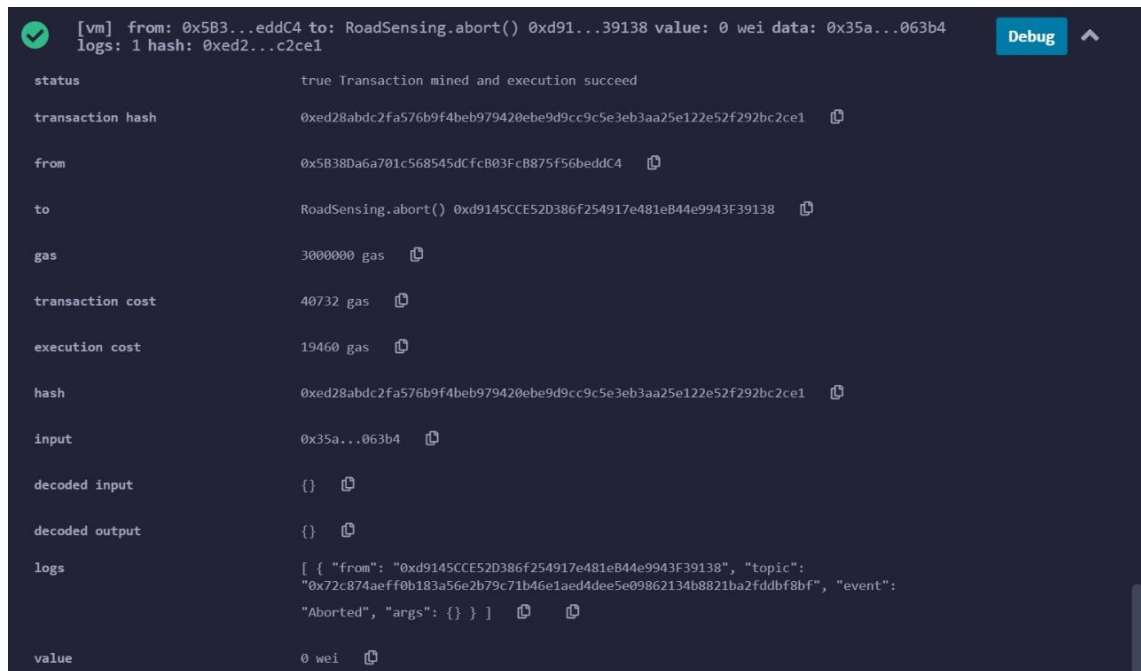


Figure A.4: Abortion of task by the requester

REFERENCES

- [1] J. Huang et al., "Blockchain-Based Mobile Crowd Sensing in Industrial Systems," 2020 in *IEEE Transactions on Industrial Informatics*, vol. 16, no. 10, pp. 6553-6563
- [2] L. K. Shien and M. M. Singh, "A Secure Mobile Crowdsensing (MCS) Location Tracker for Elderly in Smart City," 2017 in *AIP Conference Proceedings*, vol. 1, AIP Publishing, p. 020085.
- [3] Prakash, A. M. "Dynamic Evolutionary Information Diffusion over Mobile Social Networks", 2016, *ARNP Journal of Engineering and Applied Sciences*, 11(19), pp. 1145711464.
- [4] Prakash, A. M., Kamal, A., Manisha, A., "QR-Code scanner based vehicle sharing", 2018, *ARNP Journal of Engineering and Applied Sciences*, 13(10), pp. 3441-3448.
- [5] C. Luo et al., "Predictable Privacy-Preserving Mobile Crowd Sensing: A Tale of Two Roles," 2019, in *IEEE/ACM Transactions on Networking*, vol. 27, no. 1, pp. 361-374
- [6] Toch, Eran, Crowdsourcing privacy preferences in context-aware applications, 2014, *Personal and Ubiquitous Computing*. vol. 18. 10.1007/s00779-012-0632-0.
- [7] Kim, M.; Kim, Y. Multi-Blockchain Structure for a Crowdsensing-Based Smart Parking System, 2020 *Future Internet*, vol. 12, 90.
- [8] M. Li et al., "CrowdBC: A Blockchain-Based Decentralized Framework for Crowdsourcing," 2019, in *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 6, pp. 1251-1266
- [9] S. Zou, J. Xi, H. Wang and G. Xu, "CrowdBLPS: A Blockchain-Based Location-Privacy-Preserving Mobile Crowdsensing System," 2020, in *IEEE Transactions on Industrial Informatics*, vol. 16, no. 6, pp. 4206-4218
- [10] Jinbo Xiong, Rong Ma, Lei Chen, Youliang Tian, Li Lin, Biao Jin, "Achieving Incentive, Security, and Scalable Privacy Protection in Mobile Crowdsensing Services", 2018, *Wireless Communications and Mobile Computing*
- [11] L. Xiao, D. Jiang, D. Xu, W. Su, N. An and D. Wang, "Secure mobile crowdsensing based on deep learning," 2018, in *China Communications*, vol. 15, no. 10, pp. 1-11

- [12] Y. Zhang and B. Kantarci, "Invited Paper: AI-Based Security Design of Mobile Crowdsensing Systems: Review, Challenges and Case Studies," 2019 IEEE International Conference on Service-Oriented System Engineering (SOSE), San Francisco, CA, USA, 2019, pp. 17-1709
- [13] Pius Owoh N, Mahinderjit Singh M. SenseCrypt: A Security Framework for Mobile Crowd Sensing Applications. *Sensors (Basel)*. 2020; 20(11):3280
- [14] Tao Peng, Jierong Liu, Guojun Wang, Qin Liu, Jianer Chen, Jiawei Zhu, "A User-Defined LocationSharing Scheme with Efficiency and Privacy in Mobile Social Networks", 2020, *Scientific Programming*, vol. 2020, Article ID 7832875, 13 pages
- [15] S. Gisdakis, T. Giannetsos and P. Papadimitratos, "Security, Privacy, and Incentive Provision for Mobile Crowd Sensing Systems," 2016, in *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 839-853
- [16] K. AlJemy, M. AlAnazi, M. AlSofiry and A. Baig, "Improving IoT Security Using Blockchain," 2019 IEEE 10th GCC Conference & Exhibition (GCC), Kuwait, Kuwait, pp. 1-6

ORIGINALITY REPORT

3%

SIMILARITY INDEX

1%

INTERNET SOURCES

2%

PUBLICATIONS

2%

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to Bahcesehir University

Student Paper

1%

2

Submitted to University of Greenwich

Student Paper

1%

3

Shihong Zou, Jinwen Xi, Honggang Wang, Guoai Xu. "CrowdBLPS: A Blockchain-Based Location-Privacy-Preserving Mobile Crowdsensing System", IEEE Transactions on Industrial Informatics, 2020

Publication

<1%

4

www.hindawi.com

Internet Source

<1%

5

Submitted to American Public University System

Student Paper

<1%

6

www.mdpi.com

Internet Source

<1%

7

Zhu Sisi, Alireza Souri. "Blockchain technology for energy – aware mobile crowd sensing approaches in Internet of Things",

<1%

Transactions on Emerging
Telecommunications Technologies, 2021

Publication

8

Nsikak Pius Owoh, Manmeet Mahinderjit Singh. "SenseCrypt: A Security Framework for Mobile Crowd Sensing Applications", Sensors, 2020

Publication

<1%

9

Fatima Zahrae Chentouf, Said Bouchkaren. "Chapter 4 Blockchain for Cybersecurity in IoT", Springer Science and Business Media LLC, 2021

Publication

<1%

10

Vrushali Gajanan Kadam, Sharvari Chandrashekhar Tamane, Vijender Kumar Solanki. "chapter 1 Smart and Connected Cities Through Technologies", IGI Global, 2019

Publication

<1%

Exclude quotes

On

Exclude bibliography

On

Exclude matches

< 10 words

Format - I

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

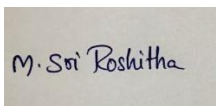
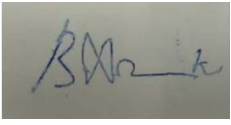
(Deemed to be University u/s 3 of UGC Act, 1956)

Office of Controller of Examinations

REPORT FOR PLAGIARISM CHECK ON THE DISSERTATION/PROJECT REPORTS FOR UG/PG PROGRAMMES

(To be attached in the dissertation/ project report)

1	Name of the Candidate (IN BLOCK LETTERS)	ANKITA PAL M.SRI ROSHITHA
2	Address of the Candidate	22 AASHNA RESIDENCY, LAD SOCIETY ROAD, VASTRAPUR, AHMEDABAD, GUJARAT - 380015 DOOR NUM : 1-113/3, PURUSHOTHPALLI, NIDADAVOLE ANDHRA PRADESH-534302 Mobile Number : 7490054195, 7901237436
3	Registration Number	RA1711003010611 RA1711003010276
4	Date of Birth	24/11/1999 23/08/2000
5	Department	COMPUTER SCIENCE AND ENGINEERING
6	Faculty	M.ARUL PRAKASH
7	Title of the Dissertation/Project	SECURE MOBILE COWD SENSING USING BLOCKCHAIN
8	Whether the above project/dissertation is done by	Individual or group : (Strike whichever is not applicable) a) If the project/ dissertation is done in group, then how many students together completed the project : 2 b) Mention the Name & Register number of other candidates : NA
9	Name and address of the Supervisor / Guide	MR. ARUL PRAKASH 9003600937 Mail ID : Mobile Number :
10	Name and address of the Co-Supervisor / Co- Guide (if any)	NA Mail ID : Mobile Number :

11	Software Used	TURNITIN		
12	Date of Verification	30/05/2021		
13	Plagiarism Details: (to attach the final report from the software)			
chapter	Tit1e of the Chapter	Percentage of similarity index (Including self citation)	Percentage of Similarity index {Excluding self citation}	% of plagiarism after excluding Quotes, Bibliography, etc.,
1	INTRODUCTION	<1%		
2	LITERATURE SURVEY	<1%		
3	MOBILE CROWD SENSING	<1%		
4	BLOCK CHAIN	2%		
5	METHODOLOGY	0%		
6	RESULTS	<1%		
7	CONCLUSION	0%		
8	CODES & OUTPUTS	NA		
9	REFERENCES	NA		
Appendices				
I / We declare that the above information have been knowledge		verified and found true to the best of my / our		
 M. SRI ROSHITHA Signature of the Candidate		 ARUL PRAKASH Name & Signature of the Staff {Who uses the plagiarism check software}		
 ARUL PRAKASH Name & Signature of the Supervisor/ Guide		Name & Signature of the Co-Supervisor/Co-Guide		
 Name & Signature of the HOD				